AI Usage Report

AI Tool Selected

AI Tool: ChatGPT

ChatGPT was used as an AI-assisted engineering tool for DBC generation, signal design, Python SocketCAN transmitter development, troubleshooting, validation, documentation, and final DBC review.

The generated DBC and source code were tested using the actual SocketCAN `vcan0` network and verified using `candump`, `cantools`, and SavvyCAN.

Prompts Used

The main prompts used were:

- Create a DBC for Vehicle Speed, Engine RPM, Coolant Temperature, Fuel Level, and Battery Voltage with appropriate CAN IDs, signal positions, scaling, offsets, ranges, and units.
- Create a Python SocketCAN transmitter using `python-can` and `vcan0`.
- Review and validate the DBC against the transmitter.
- Modify a DBC scaling factor and demonstrate its effect on decoded values.
- Add Ambient Temperature as a new CAN signal and update the transmitter and DBC.
- Review the final DBC for errors, naming, signal layout, scaling, and documentation.
- Need of DBC, correlation of SocketCAN.

AI-Generated Outputs

ChatGPT assisted in generating:

- CAN message and signal definitions
- DBC syntax and structure
- Python SocketCAN transmitter code
- Raw CAN data encoding and scaling logic
- DBC decoding procedures
- Ambient Temperature integration
- Validation and testing procedures
- Technical documentation

The final CAN network consisted of:

CAN ID	Message	Signals
`0x100`	VehicleStatus1	Vehicle Speed, Engine RPM
`0x101`	VehicleStatus2	Coolant Temperature, Fuel Level
`0x102`	BatteryStatus	Battery Voltage
`0x103`	AmbientStatus	Ambient Temperature

Corrections and Verification

The AI-generated content was verified through actual CAN testing. CAN IDs, signal ranges, scaling factors, offsets, and units were checked using `candump` and SavvyCAN.

During Challenge 2, the Battery Voltage scaling factor was changed from `0.01` to `0.02` to demonstrate the effect of an incorrect DBC definition. The original scaling was then restored.

Ambient Temperature was added as CAN ID `0x103` and successfully verified through raw CAN traffic and DBC decoding.

The final DBC review also identified a missing cycle-time entry for `0x103`, which was corrected by adding:

BA_ "GenMsgCycleTime" BO_ 259 100;

Final Observations

AI significantly reduced the time required for DBC creation, coding, debugging, and documentation. However, the generated content required practical verification because incorrect DBC definitions can produce incorrect engineering values even when CAN communication is functioning correctly.

The project demonstrated that AI is useful as an engineering assistance and review tool, while final validation should always be performed using actual CAN communication and decoding results.